

Tour — Reinforcing Recursive Language Models (learning-map companion document)

Pablo Leyva

May 15, 2026

A guided walk through the learning-map for the alphaXiv blog *Reinforcing Recursive Language Models*. Read the paper graph plus improvements graph plus foundation prereqs in the order given here.

1 Reader’s contract

Who this is for. A reader at the level of an upper-undergraduate machine learning student — comfortable with PyTorch, has seen REINFORCE / PPO once, has read a transformer-pre-training tutorial. The “advanced” concept pages assume the reader can manipulate score-function estimators and clipped surrogates without consulting a textbook.

Expected reading time. 3–4 hours for the paper graph plus 1–2 hours for the improvements graph plus 30 minutes for the proofs — roughly one full day of focused study.

Math-foundations entry point. If the reader is not yet comfortable with REINFORCE / PPO / GRPO at the equation level, start at **first-principles-to-llms** Chapter 31 and work backwards. If the reader has not yet built a transformer LM end-to-end, start at Chapter 27. If the MDP framing is unfamiliar, start at Chapter 29.

Pacing convention. Each prereq is tagged **skim** (5–10 min refresh), **read** (30–45 min careful read), or **drill** (1–2 h with paper-and-pencil derivations).

2 Foundations walk (chain prereqs)

Topologically-ordered foundation prereqs from the chain repository ([pleyva2004/first-principles-to-llms](#))

1. **Ch 25 — Causal LM as MDP (skim).** Every node in the RLM tree is one Ch 25 MDP; nothing in the paper changes that, only the inter-node structure.
2. **Ch 27 — Tiny GPT pre-training (skim).** The policy π_θ is just an autoregressive transformer; the SFT phase reuses Ch 27’s NTP loss.
3. **Ch 28 — SFT, RLHF/PPO, DPO (read).** The cold-start SFT of concept 14 is a direct lift, and the per-node loss of concept 7 is PPO. If the reader has not done PPO end-to-end before, this is the chapter to spend time on.
4. **Ch 29 — MDP foundations (read).** The trajectory tree of concept 5 is a tree of MDP rollouts; the Bellman setup is what gets lifted to SMDPs in improvement 104.
5. **Ch 30 — Max-ent + soft Bellman (skim).** Optional — gives a principled way to add an entropy bonus to the per-node loss; not needed for understanding the paper, but useful for extensions.

6. **Ch 31 — Policy gradient, GRPO, RLHF/DPO bridge (drill).** This is *the* chapter the paper extends. The baseline-doesn’t-bias proof is what makes inheritance unbiased; the GRPO sequence-level credit-assignment trick is what gets lifted to trees. If the reader does nothing else from the chain, drill this.

3 Paper concepts walk

Topologically-ordered list of the 14 paper concepts (see `paper/README.md` for the DAG):

1. Concept 1 — Recursive Language Model: what an RLM rollout *is*.
2. Concept 2 — Python REPL environment: the REPL that hosts every node.
3. Concept 3 — Root rollout: the only node that gets reward.
4. Concept 4 — Child rollout (`rlm_query`): what the recursion primitive produces.
5. Concept 5 — RLM trajectory tree: the full session is a tree.
6. Concept 6 — Shared policy: one θ for both roles.
7. Concept 7 — PPO clipped surrogate: the per-node atom.
8. Concept 8 — GRPO advantage: group-relative baseline.
9. Concept 9 — Advantage inheritance: child’s $A := A_g$.
10. Concept 10 — $1/k_g$ averaging: balancing trick.
11. Concept 11 — Recursive subtree loss: depth-agnostic loss.
12. Concept 12 — Evidence selection task: the workload.
13. Concept 13 — Rubric LLM-judge reward: the training signal.
14. Concept 14 — Cold-start SFT: bootstrap before RL.

Reading strategy. Concepts 1–6 are scaffolding; spend 10–15 minutes each. Concepts 7–11 are the load-bearing math; spend 30–45 minutes each, run the corresponding `code/` files, and verify the printed output matches the reader’s mental model. Concepts 12–14 are the experimental setup; skim unless reproducing.

4 Improvements walk

Four improvement proposals — two PROOF-validated math results and two MEASUREMENT-validated experiments. The DAG is in `improvements/README.md`.

101. Tighter unbiasedness theorem for advantage inheritance — PROOF

The blog asserts inheritance is “unbiased in the same sense as GRPO” but does not formalise the assumption. The proposed theorem makes the conditional-independence assumption (*) precise, proves unbiasedness under it, and quantifies bias when it fails (e.g., tool-use chains where child A’s output is child B’s input). Validation: `proofs/inheritance-unbiasedness.tex`.

102. Per-child local-baseline variance reduction — MEASUREMENT

Subtract a per-state baseline $b(s_v)$ from the inherited advantage before applying the surrogate. Standard control-variate trick; under (*) it is exactly unbiased and reduces vari-

ance whenever the baseline correlates with the score-function-weighted advantage. Validation: `improvements/local-baseline.py`.

103. k_g scale-law experiment — MEASUREMENT

Sweep $k_g \in \{1, 2, 4, 8, 16, 32\}$ at fixed compute budget; expect an interior optimum where per-parent reward signal and per-step batch diversity are balanced. Validation: `improvements/k-scale-sweep.py`.

104. RLM children as options (Sutton-Precup-Singh) — PROOF

Map `rlm_query` calls to options $o = (I, \pi_o, \beta)$, lift the whole RLM training problem to a semi-Markov decision process, and import SMDP Q-learning convergence (Sutton-Precup-Singh 1999, Theorem 1) under the natural conditions. The mapping is mostly a translation. Validation: `proofs/rlm-as-options.tex`.

5 What to do next

1. **Reproduce the toy bandit reward-variance gain.** Run `python3 improvements/local-baseline.py` verify the printed ratio is below 1.0. If not, the local baseline is mis-specified.
2. **Run the k_g sweep on a non-toy task.** The bandit interior-optimum is a small effect. Re-run the sweep on a real arxiv-paper retrieval workload (or one of the reader's own) to see if the optimum is at $k = 4$, $k = 16$, or larger. The answer determines deployment cost.
3. **Read `proofs/rlm-as-options.tex` and identify the most promising corollary to prove next.** The SMDP framing imports a lot of theory for free; the question is which corollary (interruptibility, intra-option learning, ...) is most actionable for the next paper. Pick one and write a 5-page extension.